6/10/2026

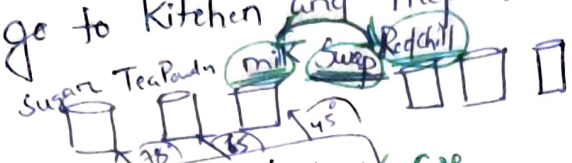
What is AI?

mimicing the human behaviour using machine

Create Robot - exactly similar behaviour in terms of understanding as those of humans.

AI ≠ Automation

Task → go to kitchen and Prepare Tea



Automate a bot (fail)
(Set of rules)

AI : AI tries to solve the Problem of Uncertainity. (can change)



LR
LR
SVM
DT
KNN
⋮
} ML Solve the Problem of Uncertainity

DL : (Deep Learning)

↳ it has neurons.
↳ Subset of ML

ML → ML focuses on Structured data. (Tabular)

DL → focuses on Both Structured and Unstructured data.

AI → ML → DL
- → Supervised (Label data) — have Target
- → Unsupervised (Non-label-data) — No Target.

Supervised DL Models

- Computer Vision Problems
- Natural Language Processing (NLP)

Computer Vision

- Object detection
- Object Classification
- object tracking
- Image Segmentation (classification based on Pixel)
  eg: Night Vision Camera (based on Pixel intensity)

Natural Language Processing

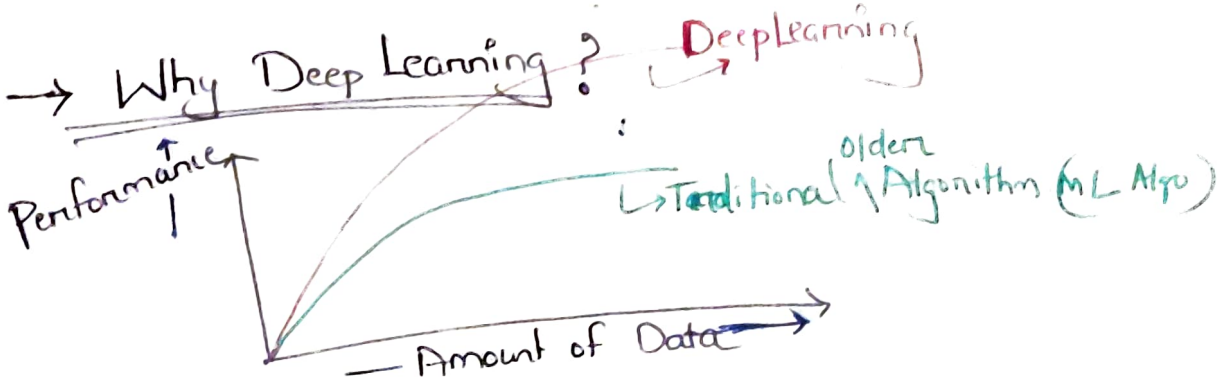
Words are having feeling → context, Sentiment

- Text Generation
- Summarization
- Classification
- Sentiment Analysis
- Machine Translation

UnSupervised DL models

- GAN : generates input by Synthetic
  (Just like SMOTE — create numerical data
  GAN — creates Synthetic data for Images.
- Autoencoders : Create Hidden features from data

→ Why Deep Learning ?



- Facebook 

Suggest friends (Deep Learning)

facebook has managed to get an accuracy of 97.25% where as human has managed to be 97.6% ← nearly equal

→ Google data Center



Server (many requests / per Second)

by the help of AI Power Consumption of Cooling System is reduced to 50% of cooling Cost :

As DL able to Predict Conried Load at Certain time.

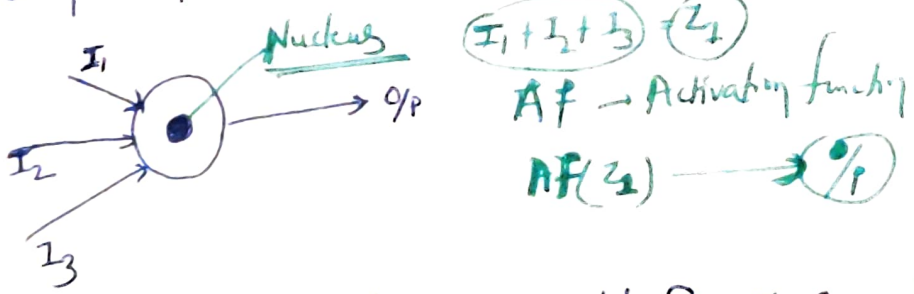
{ 50% is the 15% of Google Total Expenses }

→ DL is Computationally Expensive.
So for small data set Traditional ML doing good Job.

→ DL is the heart of AI.

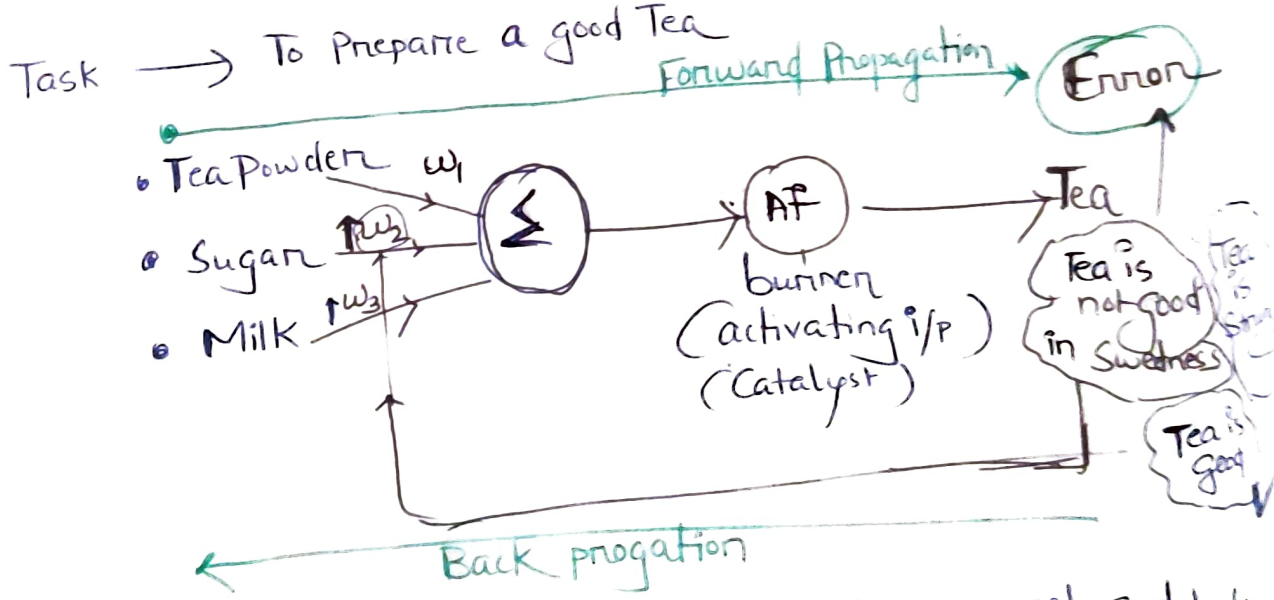
Inspiration of deep learning

→ First artificial neuron in 1957 called
→ Perceptron



Perceptron :- A probabilistic model for Information Storage and organization in brain.

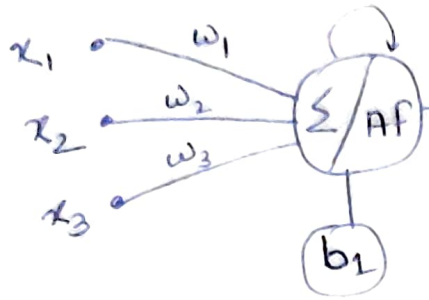
1. what is perceptron
2. Multilayer perceptron
3. Forward Propagation
4. Actifaction function
5. Math (derivatives), chain rule of differentiation
6. Backprogation
7. Types of Activation function
8. Weight Initialization Technique
9. Optimizers
10. Batch Normalization



- we increase on decrease the weight unless we get good taste.

1 iteration = FP + (Error Calculation) + BP

- At first, weights are initalized Randomly.



error

- weights are randomly initialized
- bias is added (To Prevent 0 Condition)
- w_1, w_2, w_3 } Provides amount to i/p

$$z = x_1w_1 + x_2w_2 + x_3w_3 + b_1$$

$$\hat{y} = Af(z)$$

error = Tea is not good

★ Weight Updation Rule for Perceptron (In general)

$$[w_{new}] = [w_{old}] - \eta \left[\text{Rate of change of error w.r.t change in weight}\right]$$

η - Learning rate

→ Moto is to reduce $(y - \hat{y})$ → error

actual Prediction

→ get Optimum weights.

Disadvantage of using a Perceptron (Single neuron)

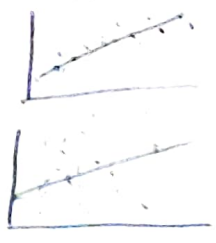
$$z = x_1w_1 + x_2w_2 + x_3w_3 + b_1$$

$$m_1x_1 + m_2x_2 + m_3x_3 + c$$ — ? equation of Straight Line $(y = mx + c)$.

equation of hyperplane

→ In Linear regression, we tring to fit a line/plane/hyperplane (2D / 3D / 4D↑) So that the MSE is Lowest.

→ But Problem with Linear Regression model was, that it works well only if your data is Linear in nature.



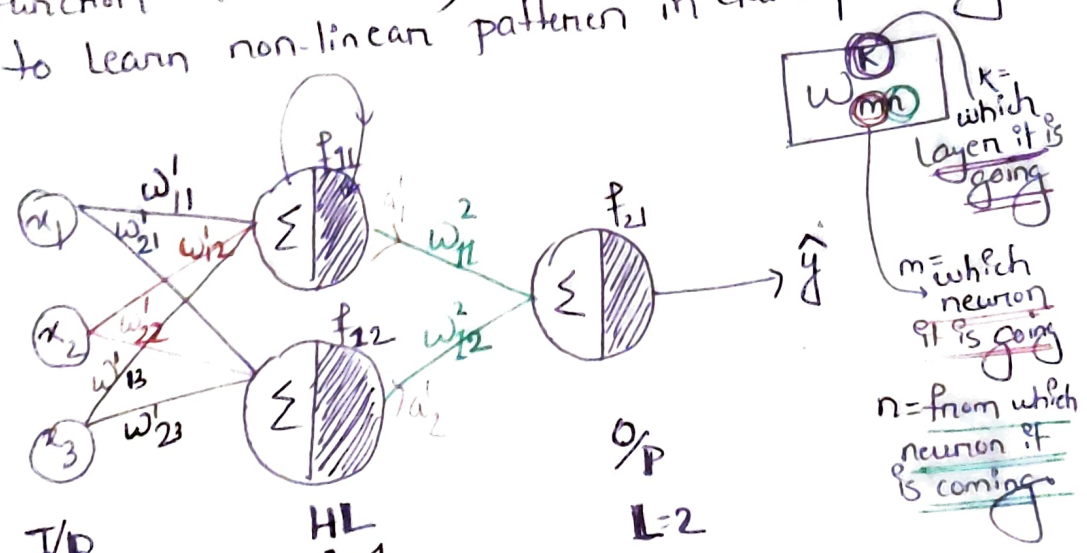
→ Since $Z = w_1x_1 + w_2x_2 + w_3x_3 + b$ is an equation of linear regression, it will only work if the data is linearly distributed.

→ But in real world, you will not see linear data that much often.

To Overcome this issue — Multi Layer Perceptron (MLP)

→ MultiLayer Perceptron

Even through Activation function (Af) make the output function non-linear, One neuron is not sufficient to learn non-linear pattern in data perfectly



Neuron

f_{mn}

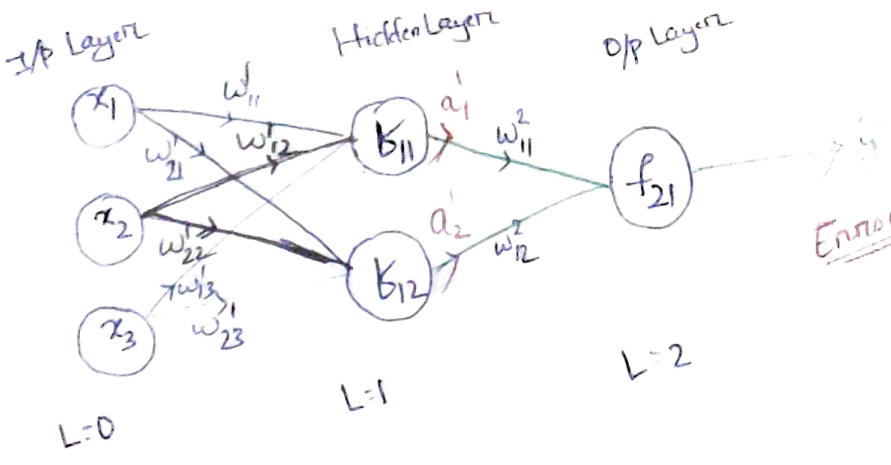
m = which Layer

n = which # of neuron

→ Training a neuron - adjusting the weight such that error should be minimized.

→ Rate of change of a function w.r.t. a Variable

→ Backpropagation (carries all the gradient Value. (error gradient)



f_{11} → 1st layer's, 1st neuron

f_{12} → 1st Layer's 2nd Neu

b_{21} – 2nd Layer's 1st neuron

Step 1 : (At Layer 1)

1st layer 1st neuron $z^1_1 = x_1 w^1_{11} + x_2 w^1_{12} + x_3 w^1_{13} + b_1$

$\Rightarrow a^1_1 = \sigma(z^1_1)$

2nd neuron $z^1_2 = x_1 w^1_{21} + x_2 w^1_{22} + x_3 w^1_{23} + b_2$

$\Rightarrow a^1_2 = \sigma(z^1_2)$

Step 2 : (At Output Layer / layer 2)

$z^2_1 = a^1_1 w^2_{11} + a^1_2 w^2_{12} + b_3$

$\hat{y} = \sigma(z^2_1)$

Step 3 : (Error Calculation)

error = Actual − Prediction

Step 4 : (Backpropagation & Weight Updation)

$w_{new} = w_{old} - \eta \left[\text{rate of change of error w.r.t change in weight} \right]$

gradients

SGD vs GD

The core difference between SGD and GD is how many training examples they use to update the model parameter in each step.

While GD (often called Batch GD) calculates the gradients using the entire dataset before making a Single Update, SGD updates the parameters after looking at just One random training example.

using Stochastic Gradient Descent

Basics of Mathematics

Chapter 1 - Vector

What is a Vector?

→ Quantity with magnitude and direction.

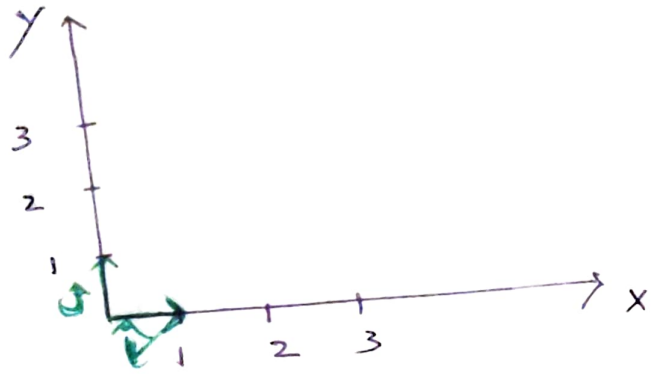
- Velocity, Force, acceleration, weight

What is Scalar?

→ Quantity with only magnitude.

- Speed, length/distance, mass, Volume

{ Time is Scalar or Vector? }

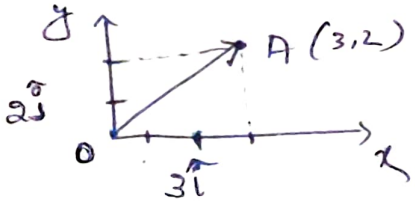


unit Vector → Vector with magnitude = 1 with direction

Unit Vector in x direction $\hat{i}$

Unit Vector in y direction $\hat{j}$

$3\hat{i}$ → A vector having 3 magnitude in x axis direction



$\overrightarrow{OA} = 3\hat{i} + 2\hat{j}$

Magnetude of $|\overrightarrow{OA}|$

$= \sqrt{3^2 + 2^2}$

$= \sqrt{9+4} = \sqrt{13}$

→ A unit Vector in the direction of $\overrightarrow{OA}$

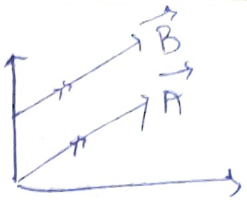
$\overrightarrow{OA} = |\overrightarrow{OA}| * \hat{a}$

$\Rightarrow \hat{a} = \frac{\overrightarrow{OA}}{|\overrightarrow{OA}|} = \frac{3\hat{i} + 2\hat{j}}{\sqrt{13}} = \frac{3\hat{i}}{\sqrt{13}} + \frac{2\hat{j}}{\sqrt{13}}$

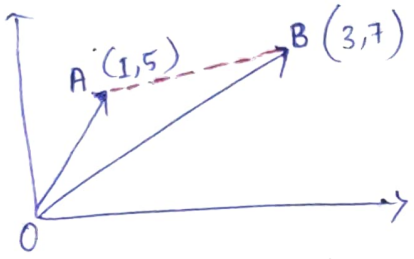
unit Vector

Condition for two vectors to be same

- Both must have equall magnitude
- Both must have same direction



Distance between two Vectors



$\overrightarrow{OA} = 1\hat{i} + 5\hat{j}$

$\overrightarrow{OB} = 3\hat{i} + 7\hat{j}$

$\overrightarrow{AB}$ = final − Source

$\overrightarrow{AB}$ = [redacted]

$= (2\hat{i} + 2\hat{j})$

Chapter 2 - Differentiation

→ It measures rate of change of a Variable wrt change in other variable.

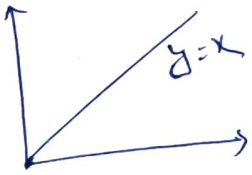
function (x)

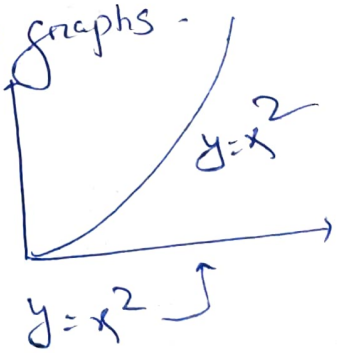
$y = x$: The value of y is dependent on the value of x

$x=1$	$x=2$	$x=3$
$y=1$	$y=2$	$y=3$

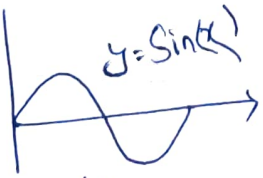
every function is depends on its variables & every function can be denoted graphically / represents graphs.

$y = x$ →





$y = \sin x$ →



Differentiation $\frac{dy}{dx} = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x}$

$\frac{dy}{dx} = \frac{\text{change in } y}{\text{Small change in } x}$

$y = x^2 + 2$

@ $x_0 = 1$

$y_0 = 1 + 2 = 3$

@ $x_1 = 1.000001$

$\Delta x = 1.000001 - 1$

$= 0.000001$

$y_1 = (1.000001)^2 + 2$

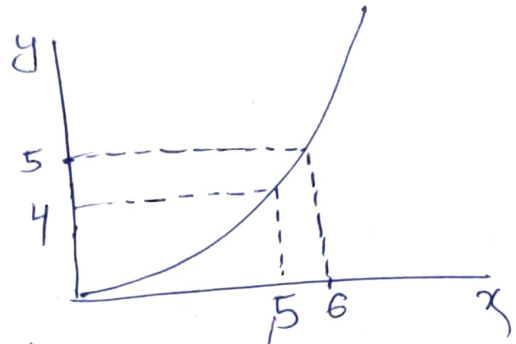
$\Delta y = y_1 - y_0$

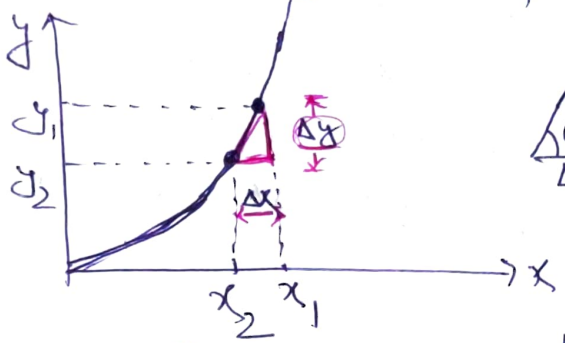
Differentiation in mathematics is the Process of finding derivatives of a function, which measures how a dependent variable changes when its independent variable changes (the instantaneous rate of change or Slope of a Curve at a specific point).

$$\left(\frac{f(x+h) - f(x)}{h}\right)$$

$$y = x^2$$

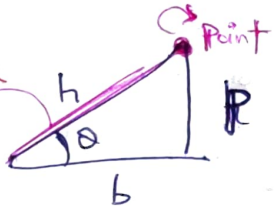
$$\frac{dy}{dx} = \lim_{x \to 0} \frac{\Delta y}{\Delta x}$$







$$\frac{dy}{dx} = \frac{\Delta y}{\Delta x}$$



$\tan\theta$ ← Slope of Tangent line

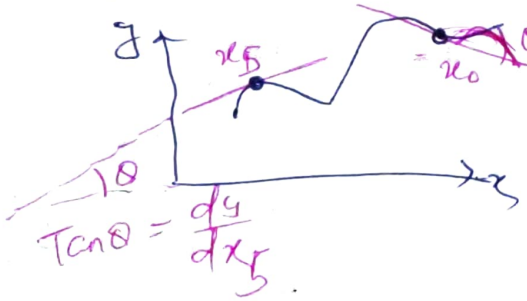
$$\tan\theta = \frac{P}{b}$$

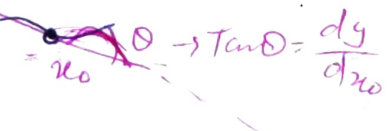
$$\sin\theta = \frac{P}{h}$$

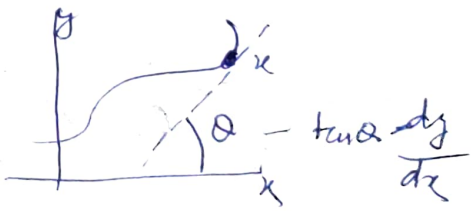
$$\cos\theta = \frac{b}{h}$$

$$\frac{dy}{dx} = \frac{\Delta y}{\Delta x} = \tan\theta$$

⇒ for any function the rate of change of it wrt to dependent variable at a point is its Tangent Value.







$\frac{dy}{dx}$ = Gradients = Tangents

$W_{new} : W_{old} - \eta$ [rate of change of error wrt change in weight]

↓

rate of change of function (error) wrt its variable (weight)

Common Formulas

$y = x^2 \Rightarrow \frac{dy}{dx} = 2x$ $\quad \left\{ \frac{d(x^n)}{dx} = n x^{n-1} \right\}$

$y = e^x \Rightarrow \frac{dy}{dx} = e^x$ $\quad \left\{ \frac{d(e^x)}{dx} = e^x \right\}$

$y = 5 \Rightarrow \frac{dy}{dx} = 0$ $\quad \left\{ \frac{d}{dx}(\text{const}) = 0 \right\}$

$y = x^2 + x^3 \Rightarrow \frac{dy}{dx} = 2x + 3x^2$

$y = (x+1)(x^2) \Rightarrow \frac{dy}{dx} = u\frac{dv}{dx} + v\frac{du}{dx}$

(u: $x+1$, v: x^2)

$\left\{ \frac{d(1st \times 2nd)}{dx} = 1st \frac{d(2nd)}{dx} + 2nd \left(\frac{d}{dx} 1st\right) \right\}$

$\Rightarrow \frac{dy}{dx} = (x+1)\frac{d(x^2)}{dx} + x^2 \frac{d(x+1)}{dx}$

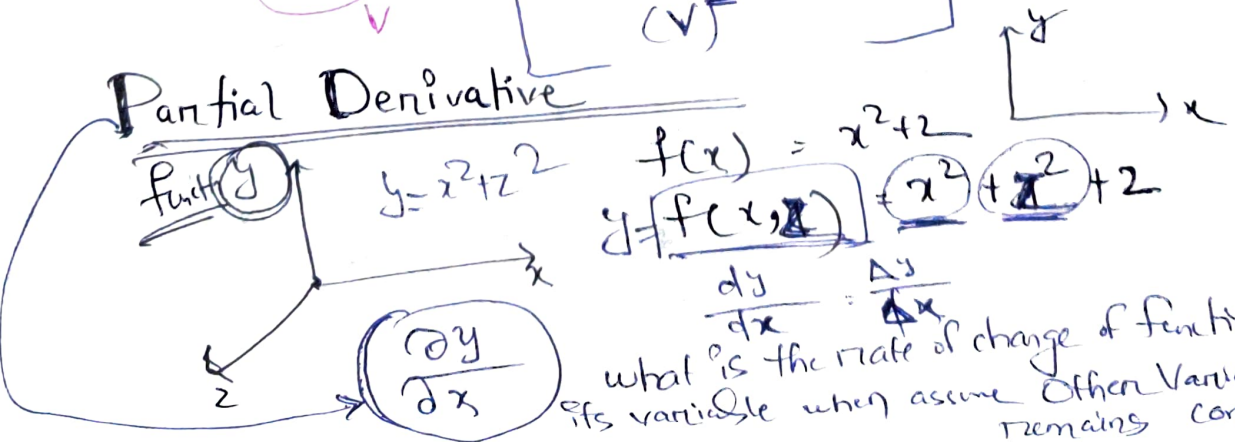
$= (x+1)2x + x^2(1+0)$

$= 2x^2 + 2x + x^2$

$= 3x^2 + 2x$

$y = \frac{(x+1)}{(x^2)} \Rightarrow \left[\frac{\left(v\frac{du}{dx} - u\cdot\frac{dv}{dx}\right)}{(v)^2} \right]$ (u: $x+1$, v: x^2)

Partial Derivative



$f(x) = x^2 + 2$

$y = f(x, z) = x^2 + z^2 + 2$

$\frac{dy}{dx} = \frac{\Delta y}{\Delta x}$

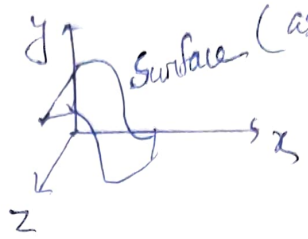
what is the rate of change of function wrt its variable when assume other variables remains constant.

$y = x^2 + z^2$

$\frac{\partial y}{\partial x}$ z=constant

$\frac{\partial y}{\partial z}$ x=constant

$y = f(x, z)$



⇒ Partial derivative : it calculates rate of change of function w.r.t. 2 or more than 2 variables.

$z = f(x, y)$

$\frac{\partial z}{\partial x}$ = change in z / Small change in x keeping y constant.

$\frac{\partial z}{\partial y}$ = change in z / Small change in y keeping x const.

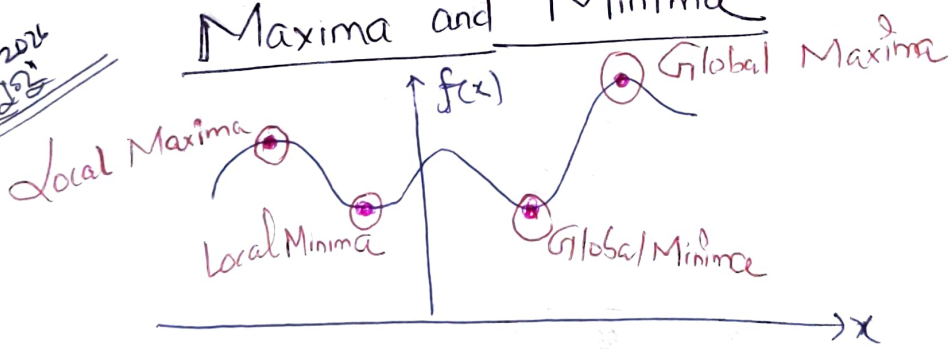
$z = x^2 + y^2$

$\Rightarrow \frac{\partial z}{\partial x} = 2x + 0$ & $\frac{\partial z}{\partial y} = 0 + 2y = 2y$

$= 2x$

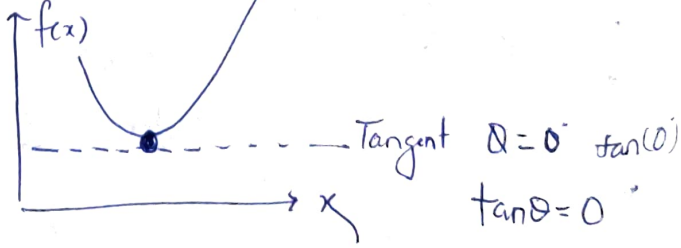
11/10/2024

Maxima and Minima



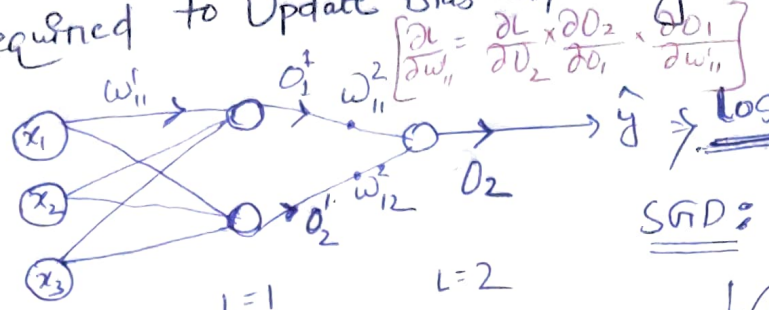
⇒ At the point of Minima or Maxima, the gradient is 0 (zero).

$\frac{dy}{dx} = 0$



Back Propagation

- It means travelling back into the Network
- Required to Update bias and Weights.



Loss = f(ω) (Actual - Predicted)

SGD: $w_{new} = w_{old} - \eta \left[\frac{dL}{dw}\right]$

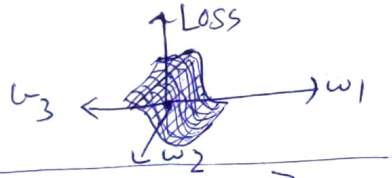
$MSE = \frac{1}{n}(y - \hat{y})^2$

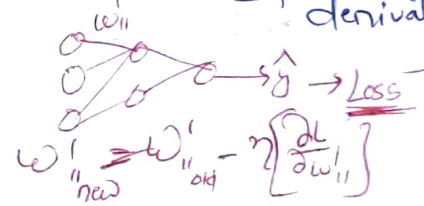
All $MSE = \frac{1}{n}(y -$

- Error is a Function of the weights Present in the System.

called: Loss function

- Since Loss function is function of many Variables
- So, let say gradient for weight w_{11}^1 → $\frac{\partial L}{\partial w_{11}^1}$ (partial derivative)

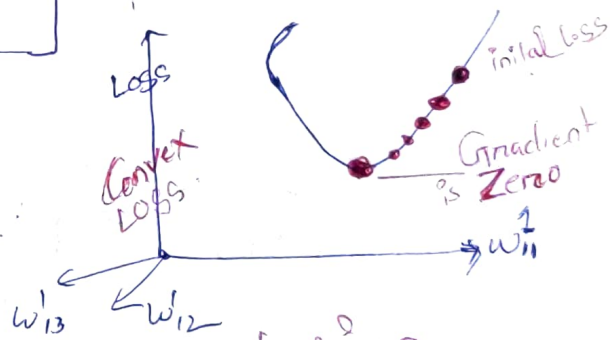




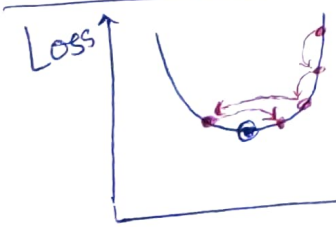
$$w_{11}^1{}_{new} = w_{11}^1{}_{old} - \eta \left[\frac{\partial L}{\partial w_{11}^1}\right]$$

η = Learning rate

η → kept as Slow as Possible (0 to 1 eg 0.01, 0.001)



- What if I make learning rate high?



Can't reach to minima

η → if very low → Convergence is going to be very Slow

η → if very high → Trying to reach minima faster, but it may miss the minima

At global minima

$\frac{\partial L}{\partial w_1^1} = 0$

$w_{new} = w_{old} - \eta \left[\frac{\partial L}{\partial w}\right]$

$w_{new} = w_{old}$

weight is not Updated then System assume Global Minima of Loss is reached. { Convergence }

Gradient Value of weight (w'_{11})

$$\left[\frac{\partial L}{\partial w'_{11}} = \frac{\partial L}{\partial O_2} \times \frac{\partial O_2}{\partial O_1} \times \frac{\partial O_1}{\partial w'_{11}}\right]$$ Chain Rule of differentiation

$\frac{\partial (\text{Sigmoid function})}{\partial O_1}$

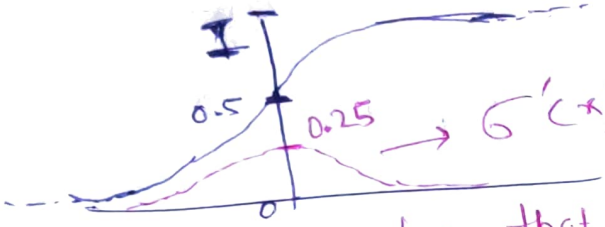
⇒ Activation function :

① Sigmoid :

→ $\sigma(x) = \frac{1}{1+e^{-x}}$

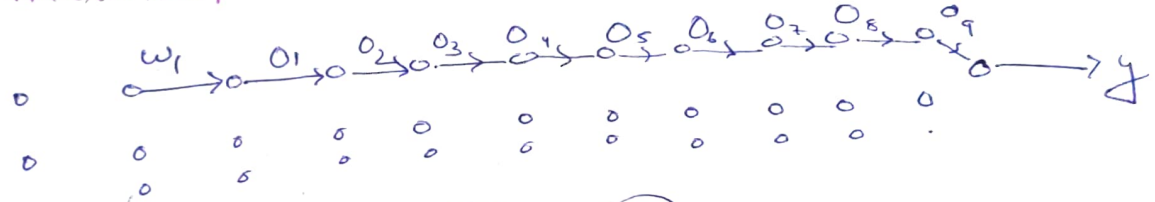
→ Ranges between (0 to 1)

⇒ Drawback in Sigmoidal function :-



→ $\sigma'(x)$ derivative of Sigmoid function

The maximum value that $\sigma'(x)$ can achieve is 0.25.



$$\left[\frac{\partial L}{\partial w_1}\right] = \frac{\partial L}{\partial O_9} \times \frac{\partial O_9}{\partial O_8} \times \frac{\partial O_8}{\partial O_7} \times \frac{\partial O_7}{\partial O_6} \times \cdots$$

$$\frac{\partial L}{\partial w_1} = \underset{\max}{(0.25} \times 0.25 \times 0.25 \times 0.25 \cdots$$

$$= 0.00000\ldots64\text{m}$$

$$w_{new} = w_{old} - \eta\left[\frac{\partial L}{\partial w_1}\right]$$

$$= w_{old} - (0.1) \times (0.00000\ldots64)$$

$$= w_{old} - \boxed{(0.000000000\,64)} \approx 0$$

⇒ $\boxed{w_{new} = w_{old}}$ Weight Update does not happen. (The Gradient itself Vanishes)

⇒ Vanishing Gradient Problem.

→ So not to use as activation function.

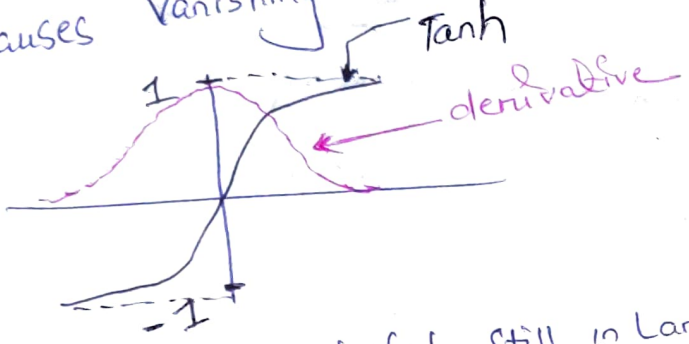
→ It is only use in the Output Layer for Binary Class Classification

(2) tanh (Hyperbolic tangent function)

↳ It Ranges from -1 to 1

↳ $\tanh = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$

→ Do not use this activation function as this also Causes Vanishing Gradient Problems.



Better than Sigmoid but still in Large network it Causes Vanishing gradient Problem. So don't use as Activation function.

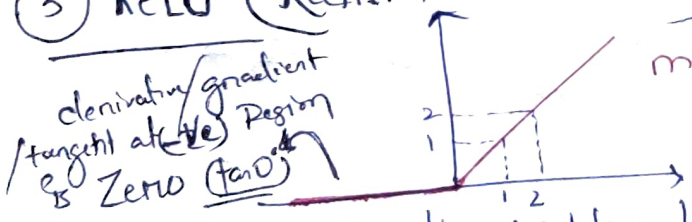
As the Maximum value of (tanh) derivative can achieve is 1

But not everytime you will be getting derivative as 1,

(0—1) → 0.4, X 0.3, X 0.2 X, 0.5 X, 0.6 = 0.0000...653 (also less)

→ Chances of vanishing Gradient

(3) ReLU (Rectified Linear Unit)



→ Used in all the hidden Layers.

→ when i/p is (-ve), the o/p is 0, when i/p is (+ve) the O/p is Same as i/p.

→ Disadvantage: Dead Neuron

→ The Only Disadvantage of using relu is dead neuron

$$\left[\frac{\partial L}{\partial w_1}\right] = \frac{\partial L}{\partial o_{12}} \times \frac{\partial o_{12}}{\partial o_{11}} \times \frac{\partial o_{11}}{\partial o_{10}} \times \frac{\partial o_{10}}{\partial o_9} \times \cdots -$$

Zero

entire becomes Zero

Suppose this neuron inputs some (-Ve) value then

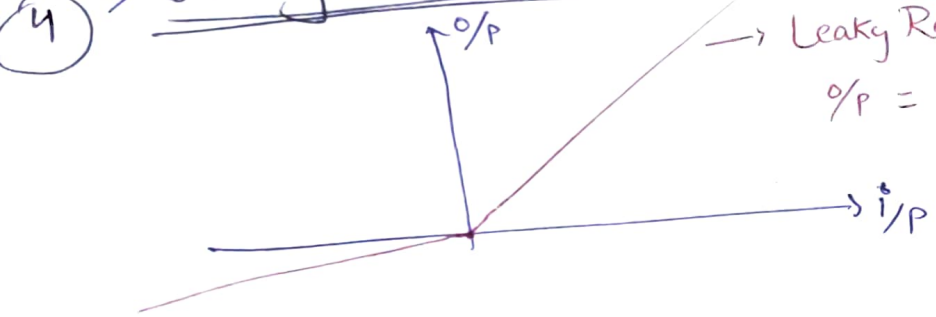
$\frac{\partial o_{11}}{\partial o_{11}} \rightarrow 0$

$\Rightarrow \left[\frac{\partial L}{\partial w_1}\right] = 0$

[[No weight Updation]]

To Solve this dead neuron Condition

4 → **Leaky ReLU** : ≈ Sckit learn Updates the ReLU → Leaky ReLU

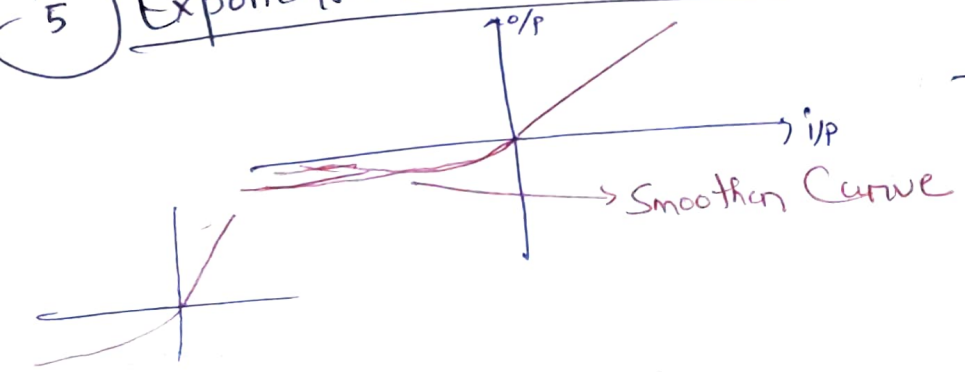


o/p = max(0.1x , x)

for (-Ve) inputs — Positive i/p

The rate of change of any Straight line is constant.

5 **Exponential Linear Unit** :



$f(x) \Rightarrow x$ → when i/p is (+Ve)

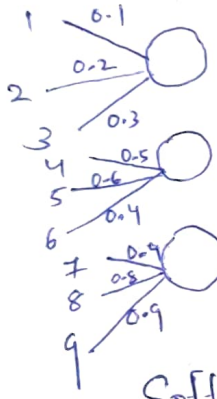
$a(e^x - 1)$ → when i/p is (-Ve)

(6) Softmax Activation function:

- Used in Multiclass Classification
- Only applied in the O/p Layer
- Caculates Probability of the O/p

→ for 3 class Classification, in O/p layer we will be using 3 neurons.

→ for Binary Class classification, it requires only 1 neurons.



$\Sigma \Rightarrow 1\times0.1 + 2\times0.2 + 3\times0.3 = 1.4$

$\Sigma \Rightarrow 4\times0.5 + 5\times0.6 + 6\times0.4 = 7.4$

$\Sigma \Rightarrow (7\times0.4 + 8\times0.8 \times 9\times0.9) = 19.4$

Softmax function Combining works with all 3 Neurons.

$$\text{Softmax}(1.4) = \left(\frac{e^{1.4}}{e^{1.4}+e^{7.4}+e^{19.4}}\right) \approx 0 \quad \text{(cat)}$$

$$\text{Softmax}(7.4) = \frac{e^{7.4}}{e^{1.4}+e^{7.4}+e^{19.4}} \approx 0 \quad \text{(dog)}$$

$$\text{Softmax}(19.4) = \frac{e^{19.4}}{e^{1.4}+e^{7.4}+e^{19.4}} \approx 0.99 \quad \text{(lion)}$$

Probabilities.

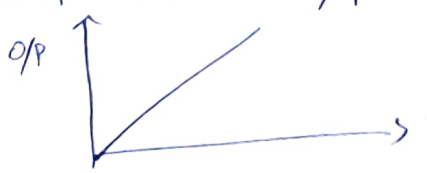
→ Out of all the Probabilities, whichever Probability is Maximum that will be final Output.

(7) Linear Activation Function:

→ No activation

→ Identity function

→ Only used in the O/p layer for Regression Problem



O/p Layer Require 1 neuron

Weight Initialization technique

→ Xavier/glorot initialization

→ He initialization
(Scientist)

① Xavier Normal

$w \sim N(0, Var(w))$
(mean, variance)

$Var(w) = \frac{\sqrt{2}}{n_{in} + n_{out}}$

② Xavier Uniform

$w \sim$ comes from Uniform distribution

$w = U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}}, +\sqrt{\frac{6}{n_{in}+n_{out}}}\right)$

① He Normal

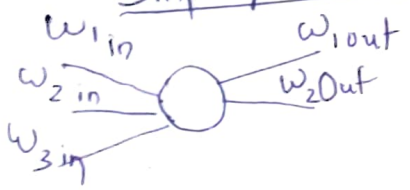
$w \sim N(0, Var(w))$

$Var(w) = \frac{\sqrt{2}}{n_{in}}$

② He Uniform

$w = U\left(-\sqrt{\frac{6}{n_{in}}}, +\sqrt{\frac{6}{n_{in}}}\right)$

3 input / 2 output



$\Rightarrow \frac{\sqrt{2}}{3+2} = \frac{\sqrt{2}}{5}$



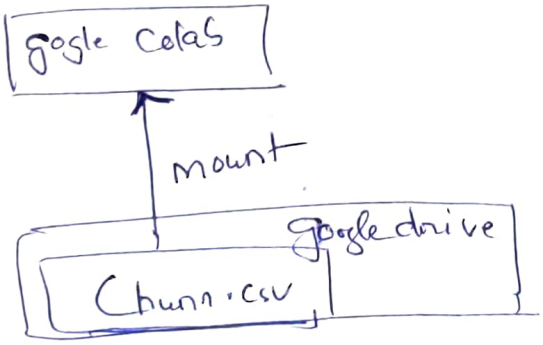
mean = 0

Variance = $\frac{\sqrt{2}}{5}$

→ all weights comes from a Normal Distribution having

→ for relu, use he Normal or Uniform

→ for Other, use Xavier Normal or Uniform



Fruits	get dummies →	apple	ball	Cucumber
apple		1	0	0
ball		0	1	0
apple		1	0	0
Cucumber		0	0	1

droping first column

```
Geography = pd.get_dummies(data.Geography, drop_first=True)
Gender    = pd.get_dummies(data.Gender, drop_first=True)
```

⇒ Curse of dimensionality

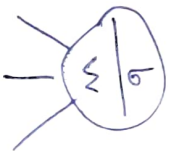
```
data = pd.concat([data, Geography, Gender], axis=1)
```

```
# Separate X, y
# train test split
# Scaling (Imp)
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

X_train.shape (7500, 11)
X_test.shape (2500, 11)

No Scaling Required → Decision Tree, Random forest, XG Boost

Scaling Required → LR, KNN, Kmeans, SVM (Distance based Algorithm)



$w_1x_1 + w_2x_2 + w_3x_3 + b$

Linear Regression

Scaling required else Some particular feature/column with high value Model biased towards this column

→ fit → Learn from data (μ, σ)

→ fit_transform → learn (μ, σ), then transform $\left(z = \frac{x_i - \mu}{\sigma}\right)$

StandardScaler
$\mu = 0$
$\sigma = 1$



Zscore

X		X_scaled
200	Zvalue →	$\left(\frac{200-\mu}{\sigma}\right)$
500		$\left(\frac{500-\mu}{\sigma}\right)$
600		$\left(\frac{600-\mu}{\sigma}\right)$
300		$\left(\frac{300-\mu}{\sigma}\right)$
100		$\left(\frac{100-\mu}{\sigma}\right)$
μ =		
σ =	Zscore →	$\mu = 0$, $\sigma = 1$

Tensorflow → framework to deals with deeplearning

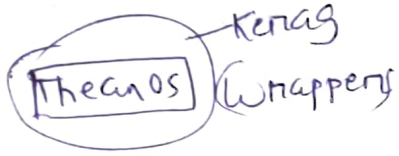
↳ google's product

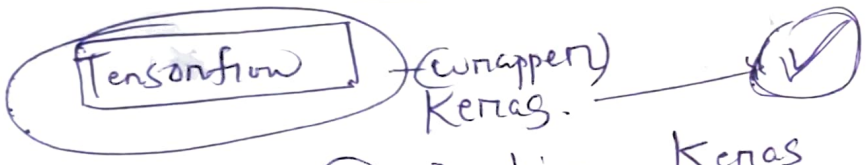
↳ it has too much Low level tweaking

Francois - scientist → Keras

Theanos — Too Complecated (Very Low Level) before Tensorflow

Keras → wrapper around Theanos.



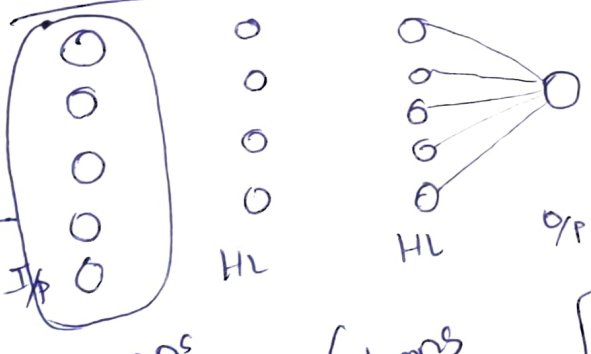


Then Google hire Keras (Francois)

from tensorflow import Keras
from Keras import tensorflow

→ Keras is a Refines Version of tensorflow

MLP → Binary Class Classification

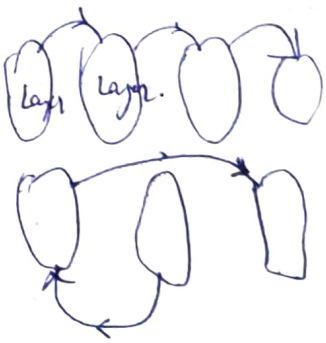


#Input Neurons = #of features/columns

Model

→ Sequential

→ Functional





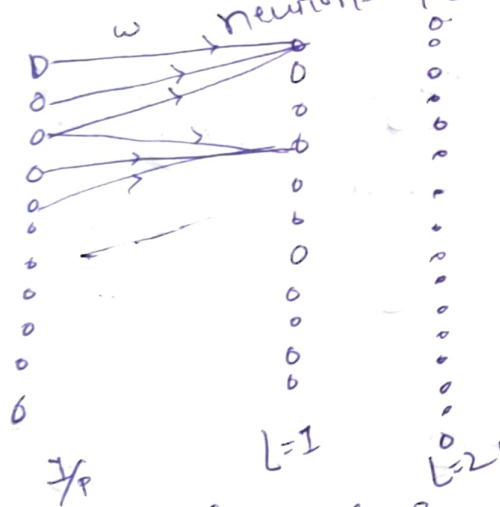
Dropout(0.3) — deactivate 30% of the neurons because Neural Network is prone to Overfitting

→ model = Sequential()

→ model.add(Dense(units=11, input_dim=11, activation='relu', kernel_initializer='he_uniform'))

How many neurons in L1

weight Initialization



→ model.add(Dense(units=16, activation='relu', kernel_initializer='he_uniform'))

→ ---------- (units=8, ----)

→ model.add(Dense(units=1, activation="Sigmoid", kernel_initializer="glorot_uniform"))

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense, Dropout

→ model.summary()



total weights = 11 × 11 = 121 + 11 (bias) = 132

Compile the Keras Model

```
model.compile( loss = "binary_crossentropy", optimizer = "adam"
             , metrics = ["accuracy"] )
```

refer Keras documentation (loss)

1. mse ← Regression
2. binary_crossentropy ← classification (Binary)
3. (Categorical_crossentropy) ← Multiclass classification (Multi)
   (Sparse_categoricalcrossentropy) or

fit the Keras model on the dataset

```
model_history = model.fit( X_train, y_train, epochs = 15
                         , validation_split = 0.33
                         , batch_size = 10 )
```

→ Complete data goes 1 epochs

33% Validation data used to evaluate / validate Performance of 33% of data at every epoch.

Batch_size

X_1	X_2	X_3	Target
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			

batch_size = 2

At a particular time 2 records will go to the Network

To Complete 1 epoch it takes 5 iterations

X_train.shape
(7500, 11)

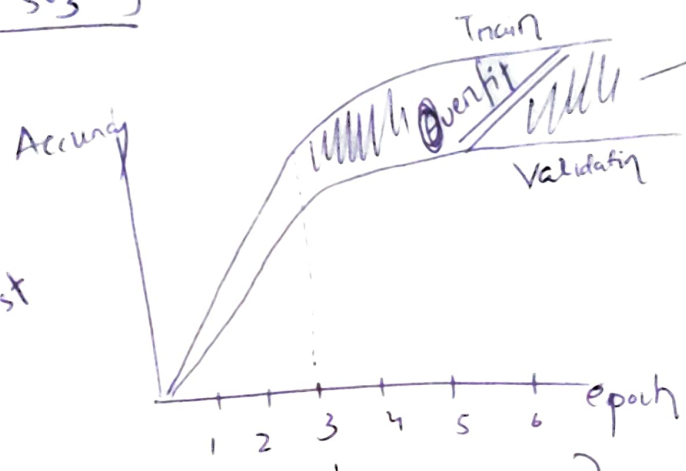
epoch = 15

batch_size = 10
↳ at One Iteration 10 data points will go

$\frac{7500}{10} = 750$ Iteration to Complete 1 Epochs

{33% Validation

Epoch 1/15
503/503



Train accuracy Increasing
but Test accuracy is
Not increasing

Overfit
↳ Good Performance Train
on
But Bad on Test

Stop at this epoch

→ Retrain the whole model with only 3 epochs

→ Similar things for Loss.

→ Losses in Keras Sequential model

→ Type Code { ANN or Regression }